



OpenShift Container Platform 4.20

Installation configuration

Cluster-wide configuration during installations

OpenShift Container Platform 4.20 Installation configuration

Cluster-wide configuration during installations

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document describes how to perform initial OpenShift Container Platform cluster configuration.

Table of Contents

CHAPTER 1. CUSTOMIZING NODES	3
1.1. CREATING MACHINE CONFIGS WITH BUTANE	3
1.1.1. About Butane	3
1.1.2. Installing Butane	3
1.1.3. Creating a MachineConfig object by using Butane	4
1.2. ADDING DAY-1 KERNEL ARGUMENTS	5
1.3. ADDING KERNEL MODULES TO NODES	6
1.3.1. Building and testing the kernel module container	7
1.3.2. Provisioning a kernel module to OpenShift Container Platform	10
1.3.2.1. Provision kernel modules via a MachineConfig object	10
1.4. ENCRYPTING AND MIRRORING DISKS DURING INSTALLATION	12
1.4.1. About disk encryption	12
1.4.1.1. Configuring an encryption threshold	13
1.4.2. About disk mirroring	15
1.4.3. Configuring disk encryption and mirroring	15
1.5. CONFIGURING A RAID-ENABLED DATA VOLUME	22
1.5.1. Configuring an Intel® Virtual RAID on CPU (VROC) data volume	25
1.6. CONFIGURING CHRONY TIME SERVICE	26
1.7. ADDITIONAL RESOURCES	28
CHAPTER 2. CONFIGURING YOUR FIREWALL	29
2.1. CONFIGURING YOUR FIREWALL FOR OPENSIFT CONTAINER PLATFORM	29
2.2. OPENSIFT CONTAINER PLATFORM NETWORK FLOW MATRIX	34
2.2.1. Base network flows	35
2.2.2. Additional network flows for OpenShift Container Platform on bare metal	40
2.2.3. Additional network flows for single-node OpenShift with other platforms	42
2.2.4. Additional network flows for OpenShift Container Platform on AWS	42
2.2.5. Additional network flows for single-node OpenShift on AWS	43

CHAPTER 1. CUSTOMIZING NODES

OpenShift Container Platform supports both cluster-wide and per-machine configuration via Ignition, which allows arbitrary partitioning and file content changes to the operating system. In general, if a configuration file is documented in Red Hat Enterprise Linux (RHEL), then modifying it via Ignition is supported.

There are two ways to deploy machine config changes:

- Creating machine configs that are included in manifest files to start up a cluster during **openshift-install**.
- Creating machine configs that are passed to running OpenShift Container Platform nodes via the Machine Config Operator.

Additionally, modifying the reference config, such as the Ignition config that is passed to **coreos-installer** when installing bare-metal nodes allows per-machine configuration. These changes are currently not visible to the Machine Config Operator.

The following sections describe features that you might want to configure on your nodes in this way.

1.1. CREATING MACHINE CONFIGS WITH BUTANE

Machine configs are used to configure control plane and worker machines by instructing machines how to create users and file systems, set up the network, install systemd units, and more.

Because modifying machine configs can be difficult, you can use Butane configs to create machine configs for you, thereby making node configuration much easier.

1.1.1. About Butane

Butane is a command-line utility that OpenShift Container Platform uses to provide convenient, short-hand syntax for writing machine configs, as well as for performing additional validation of machine configs. The format of the Butane config file that Butane accepts is defined in the [OpenShift Butane config spec](#).

1.1.2. Installing Butane

You can install the Butane tool (**butane**) to create OpenShift Container Platform machine configs from a command-line interface. You can install **butane** on Linux, Windows, or macOS by downloading the corresponding binary file.

TIP

Butane releases are backwards-compatible with older releases and with the Fedora CoreOS Config Transpiler (FCCT).

Procedure

1. Navigate to the Butane image download page at <https://mirror.openshift.com/pub/openshift-v4/clients/butane/>.
2. Get the **butane** binary:

- a. For the newest version of Butane, save the latest **butane** image to your current directory:

```
$ curl https://mirror.openshift.com/pub/openshift-v4/clients/butane/latest/butane --output butane
```

- b. Optional: For a specific type of architecture you are installing Butane on, such as aarch64 or ppc64le, indicate the appropriate URL. For example:

```
$ curl https://mirror.openshift.com/pub/openshift-v4/clients/butane/latest/butane-aarch64 --output butane
```

3. Make the downloaded binary file executable:

```
$ chmod +x butane
```

4. Move the **butane** binary file to a directory on your **PATH**.

To check your **PATH**, open a terminal and execute the following command:

```
$ echo $PATH
```

Verification steps

- You can now use the Butane tool by running the **butane** command:

```
$ butane <butane_file>
```

1.1.3. Creating a MachineConfig object by using Butane

You can use Butane to produce a **MachineConfig** object so that you can configure worker or control plane nodes at installation time or via the Machine Config Operator.

Prerequisites

- You have installed the **butane** utility.

Procedure

1. Create a Butane config file. The following example creates a file named **99-worker-custom.bu** that configures the system console to show kernel debug messages and specifies custom settings for the chrony time service:

```
variant: openshift
version: 4.20.0
metadata:
  name: 99-worker-custom
labels:
  machineconfiguration.openshift.io/role: worker
openshift:
  kernel_arguments:
    - loglevel=7
storage:
  files:
```



```
- path: /etc/chrony.conf
  mode: 0644
  overwrite: true
  contents:
    inline: |
      pool 0.rhel.pool.ntp.org iburst
      driftfile /var/lib/chrony/drift
      makestep 1.0 3
      rtcsync
      logdir /var/log/chrony
```



NOTE

The **99-worker-custom.bu** file is set to create a machine config for worker nodes. To deploy on control plane nodes, change the role from **worker** to **master**. To do both, you could repeat the whole procedure using different file names for the two types of deployments.

2. Create a **MachineConfig** object by giving Butane the file that you created in the previous step:

```
$ butane 99-worker-custom.bu -o ./99-worker-custom.yaml
```

A **MachineConfig** object YAML file is created for you to finish configuring your machines.

3. Save the Butane config in case you need to update the **MachineConfig** object in the future.
4. If the cluster is not running yet, generate manifest files and add the **MachineConfig** object YAML file to the **openshift** directory. If the cluster is already running, apply the file as follows:

```
$ oc create -f 99-worker-custom.yaml
```

Additional resources

- [Adding kernel modules to nodes](#)
- [Encrypting and mirroring disks during installation](#)

1.2. ADDING DAY-1 KERNEL ARGUMENTS

Although it is often preferable to modify kernel arguments as a day-2 activity, you might want to add kernel arguments to all master or worker nodes during initial cluster installation. Here are some reasons you might want to add kernel arguments during cluster installation so they take effect before the systems first boot up:

- You need to do some low-level network configuration before the systems start.
- You want to disable a feature, such as SELinux, so it has no impact on the systems when they first come up.

**WARNING**

Disabling SELinux on RHCOS in production is not supported. Once SELinux has been disabled on a node, it must be re-provisioned before re-inclusion in a production cluster.

To add kernel arguments to master or worker nodes, you can create a **MachineConfig** object and inject that object into the set of manifest files used by Ignition during cluster setup.

For a listing of arguments you can pass to a RHEL 8 kernel at boot time, see [Kernel.org kernel parameters](https://kernel.org/doc/Documentation/kernel-parameters.txt). It is best to only add kernel arguments with this procedure if they are needed to complete the initial OpenShift Container Platform installation.

Procedure

1. Change to the directory that contains the installation program and generate the Kubernetes manifests for the cluster:

```
$ ./openshift-install create manifests --dir <installation_directory>
```

2. Decide if you want to add kernel arguments to worker or control plane nodes.
3. In the **openshift** directory, create a file (for example, **99-openshift-machineconfig-master-kargs.yaml**) to define a **MachineConfig** object to add the kernel settings. This example adds a **loglevel=7** kernel argument to control plane nodes:

```
$ cat << EOF > 99-openshift-machineconfig-master-kargs.yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: master
  name: 99-openshift-machineconfig-master-kargs
spec:
  kernelArguments:
    - loglevel=7
EOF
```

You can change **master** to **worker** to add kernel arguments to worker nodes instead. Create a separate YAML file to add to both master and worker nodes.

You can now continue on to create the cluster.

1.3. ADDING KERNEL MODULES TO NODES

For most common hardware, the Linux kernel includes the device driver modules needed to use that hardware when the computer starts up. For some hardware, however, modules are not available in Linux. Therefore, you must find a way to provide those modules to each host computer. This procedure describes how to do that for nodes in an OpenShift Container Platform cluster.

When a kernel module is first deployed by following these instructions, the module is made available for the current kernel. If a new kernel is installed, the `kmods-via-containers` software will rebuild and deploy the module so a compatible version of that module is available with the new kernel.

The way that this feature is able to keep the module up to date on each node is by:

- Adding a `systemd` service to each node that starts at boot time to detect if a new kernel has been installed and
- If a new kernel is detected, the service rebuilds the module and installs it to the kernel

For information on the software needed for this procedure, see the [kmods-via-containers github site](#).

A few important issues to keep in mind:

- This procedure is Technology Preview.
- Software tools and examples are not yet available in official RPM form and can only be obtained for now from unofficial **github.com** sites noted in the procedure.
- Third-party kernel modules you might add through these procedures are not supported by Red Hat.
- In this procedure, the software needed to build your kernel modules is deployed in a RHEL 8 container. Keep in mind that modules are rebuilt automatically on each node when that node gets a new kernel. For that reason, each node needs access to a **yum** repository that contains the kernel and related packages needed to rebuild the module. That content is best provided with a valid RHEL subscription.

1.3.1. Building and testing the kernel module container

Before deploying kernel modules to your OpenShift Container Platform cluster, you can test the process on a separate RHEL system. Gather the kernel module's source code, the KVC framework, and the `kmod-via-containers` software. Then build and test the module. To do that on a RHEL 8 system, do the following:

Procedure

1. Register a RHEL 8 system:

```
# subscription-manager register
```

2. Attach a subscription to the RHEL 8 system:

```
# subscription-manager attach --auto
```

3. Install software that is required to build the software and container:

```
# yum install podman make git -y
```

4. Clone the **kmod-via-containers** repository:

- a. Create a folder for the repository:

```
$ mkdir kmods; cd kmods
```

-
- b. Clone the repository:

```
$ git clone https://github.com/kmods-via-containers/kmods-via-containers
```

5. Install a KVC framework instance on your RHEL 8 build host to test the module. This adds a **kmods-via-container** systemd service and loads it:

- a. Change to the **kmod-via-containers** directory:

```
$ cd kmods-via-containers/
```

- b. Install the KVC framework instance:

```
$ sudo make install
```

- c. Reload the systemd manager configuration:

```
$ sudo systemctl daemon-reload
```

6. Get the kernel module source code. The source code might be used to build a third-party module that you do not have control over, but is supplied by others. You will need content similar to the content shown in the **kvc-simple-kmod** example that can be cloned to your system as follows:

```
$ cd .. ; git clone https://github.com/kmods-via-containers/kvc-simple-kmod
```

7. Edit the configuration file, **simple-kmod.conf** file, in this example, and change the name of the Dockerfile to **Dockerfile.rhel**:

- a. Change to the **kvc-simple-kmod** directory:

```
$ cd kvc-simple-kmod
```

- b. Rename the Dockerfile:

```
$ cat simple-kmod.conf
```

Example Dockerfile

```
KMOD_CONTAINER_BUILD_CONTEXT="https://github.com/kmods-via-containers/kvc-simple-kmod.git"
KMOD_CONTAINER_BUILD_FILE=Dockerfile.rhel
KMOD_SOFTWARE_VERSION=dd1a7d4
KMOD_NAMES="simple-kmod simple-procfs-kmod"
```

8. Create an instance of **kmods-via-containers@.service** for your kernel module, **simple-kmod** in this example:

```
$ sudo make install
```

9. Enable the **kmods-via-containers@.service** instance:

```
$ sudo kmods-via-containers build simple-kmod $(uname -r)
```

10. Enable and start the systemd service:

```
$ sudo systemctl enable kmods-via-containers@simple-kmod.service --now
```

- a. Review the service status:

```
$ sudo systemctl status kmods-via-containers@simple-kmod.service
```

Example output

- `kmods-via-containers@simple-kmod.service` - Kmods Via Containers - simple-kmod
Loaded: loaded (/etc/systemd/system/kmods-via-containers@.service;
enabled; vendor preset: disabled)
Active: active (exited) since Sun 2020-01-12 23:49:49 EST; 5s ago...

11. To confirm that the kernel modules are loaded, use the **lsmod** command to list the modules:

```
$ lsmod | grep simple_
```

Example output

```
simple_procfs_kmod    16384 0
simple_kmod           16384 0
```

12. Optional. Use other methods to check that the **simple-kmod** example is working:

- Look for a "Hello world" message in the kernel ring buffer with **dmesg**:

```
$ dmesg | grep 'Hello world'
```

Example output

```
[ 6420.761332] Hello world from simple_kmod.
```

- Check the value of **simple-procfs-kmod** in **/proc**:

```
$ sudo cat /proc/simple-procfs-kmod
```

Example output

```
simple-procfs-kmod number = 0
```

- Run the **spkut** command to get more information from the module:

```
$ sudo spkut 44
```

Example output

```
KVC: wrapper simple-kmod for 4.20.0-147.3.1.el8_1.x86_64
Running userspace wrapper using the kernel module container...
+ podman run -i --rm --privileged
  simple-kmod-dd1a7d4:4.20.0-147.3.1.el8_1.x86_64 spkut 44
simple-procfs-kmod number = 0
simple-procfs-kmod number = 44
```

Going forward, when the system boots this service will check if a new kernel is running. If there is a new kernel, the service builds a new version of the kernel module and then loads it. If the module is already built, it will just load it.

1.3.2. Provisioning a kernel module to OpenShift Container Platform

Depending on whether or not you must have the kernel module in place when OpenShift Container Platform cluster first boots, you can set up the kernel modules to be deployed in one of two ways:

- **Provision kernel modules at cluster install time (day-1)** You can create the content as a **MachineConfig** object and provide it to **openshift-install** by including it with a set of manifest files.
- **Provision kernel modules via Machine Config Operator (day-2)** If you can wait until the cluster is up and running to add your kernel module, you can deploy the kernel module software via the Machine Config Operator (MCO).

In either case, each node needs to be able to get the kernel packages and related software packages at the time that a new kernel is detected. There are a few ways you can set up each node to be able to obtain that content.

- Provide RHEL entitlements to each node.
- Get RHEL entitlements from an existing RHEL host, from the **/etc/pki/entitlement** directory and copy them to the same location as the other files you provide when you build your Ignition config.
- Inside the Dockerfile, add pointers to a **yum** repository containing the kernel and other packages. This must include new kernel packages as they are needed to match newly installed kernels.

1.3.2.1. Provision kernel modules via a MachineConfig object

By packaging kernel module software with a **MachineConfig** object, you can deliver that software to worker or control plane nodes at installation time or via the Machine Config Operator.

Procedure

1. Register a RHEL 8 system:

```
# subscription-manager register
```

2. Attach a subscription to the RHEL 8 system:

```
# subscription-manager attach --auto
```

3. Install software needed to build the software:

```
# yum install podman make git -y
```

4. Create a directory to host the kernel module and tooling:

```
$ mkdir kmods; cd kmods
```

5. Get the **kmods-via-containers** software:

- a. Clone the **kmods-via-containers** repository:

```
$ git clone https://github.com/kmods-via-containers/kmods-via-containers
```

- b. Clone the **kvc-simple-kmod** repository:

```
$ git clone https://github.com/kmods-via-containers/kvc-simple-kmod
```

6. Get your module software. In this example, **kvc-simple-kmod** is used.

7. Create a fakeroot directory and populate it with files that you want to deliver via Ignition, using the repositories cloned earlier:

- a. Create the directory:

```
$ FAKEROOT=$(mktemp -d)
```

- b. Change to the **kmod-via-containers** directory:

```
$ cd kmods-via-containers
```

- c. Install the KVC framework instance:

```
$ make install DESTDIR=${FAKEROOT}/usr/local CONFDIR=${FAKEROOT}/etc/
```

- d. Change to the **kvc-simple-kmod** directory:

```
$ cd ../kvc-simple-kmod
```

- e. Create the instance:

```
$ make install DESTDIR=${FAKEROOT}/usr/local CONFDIR=${FAKEROOT}/etc/
```

8. Clone the fakeroot directory, replacing any symbolic links with copies of their targets, by running the following command:

```
$ cd .. && rm -rf kmod-tree && cp -Lpr ${FAKEROOT} kmod-tree
```

9. Create a Butane config file, **99-simple-kmod.bu**, that embeds the kernel module tree and enables the systemd service.

**NOTE**

See "Creating machine configs with Butane" for information about Butane.

```
variant: openshift
version: 4.20.0
metadata:
  name: 99-simple-kmod
  labels:
    machineconfiguration.openshift.io/role: worker 1
storage:
  trees:
    - local: kmod-tree
systemd:
  units:
    - name: kmods-via-containers@simple-kmod.service
      enabled: true
```

- 1** To deploy on control plane nodes, change **worker** to **master**. To deploy on both control plane and worker nodes, perform the remainder of these instructions once for each node type.

10. Use Butane to generate a machine config YAML file, **99-simple-kmod.yaml**, containing the files and configuration to be delivered:

```
$ butane 99-simple-kmod.bu --files-dir . -o 99-simple-kmod.yaml
```

11. If the cluster is not up yet, generate manifest files and add this file to the **openshift** directory. If the cluster is already running, apply the file as follows:

```
$ oc create -f 99-simple-kmod.yaml
```

Your nodes will start the **kmods-via-containers@simple-kmod.service** service and the kernel modules will be loaded.

12. To confirm that the kernel modules are loaded, you can log in to a node (using **oc debug node/<openshift-node>**, then **chroot /host**). To list the modules, use the **lsmod** command:

```
$ lsmod | grep simple_
```

Example output

```
simple_procfs_kmod 16384 0
simple_kmod         16384 0
```

1.4. ENCRYPTING AND MIRRORING DISKS DURING INSTALLATION

During an OpenShift Container Platform installation, you can enable boot disk encryption and mirroring on the cluster nodes.

1.4.1. About disk encryption

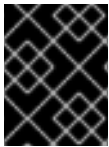
You can enable encryption for the boot disks on the control plane and compute nodes at installation time. OpenShift Container Platform supports the Trusted Platform Module (TPM) v2 and Tang encryption modes.

TPM v2

This is the preferred mode. TPM v2 stores passphrases in a secure cryptoprocessor on the server. You can use this mode to prevent decryption of the boot disk data on a cluster node if the disk is removed from the server.

Tang

Tang and Clevis are server and client components that enable network-bound disk encryption (NBDE). You can bind the boot disk data on your cluster nodes to one or more Tang servers. This prevents decryption of the data unless the nodes are on a secure network where the Tang servers are accessible. Clevis is an automated decryption framework used to implement decryption on the client side.



IMPORTANT

The use of the Tang encryption mode to encrypt your disks is only supported for bare metal and vSphere installations on user-provisioned infrastructure.

In earlier versions of Red Hat Enterprise Linux CoreOS (RHCOS), disk encryption was configured by specifying `/etc/clevis.json` in the Ignition config. That file is not supported in clusters created with OpenShift Container Platform 4.7 or later. Configure disk encryption by using the following procedure.

When the TPM v2 or Tang encryption modes are enabled, the RHCOS boot disks are encrypted using the LUKS2 format.

This feature:

- Is available for installer-provisioned infrastructure, user-provisioned infrastructure, and Assisted Installer deployments
- For Assisted installer deployments:
 - Each cluster can only have a single encryption method, Tang or TPM
 - Encryption can be enabled on some or all nodes
 - There is no Tang threshold; all servers must be valid and operational
 - Encryption applies to the installation disks only, not to the workload disks
- Is supported on Red Hat Enterprise Linux CoreOS (RHCOS) systems only
- Sets up disk encryption during the manifest installation phase, encrypting all data written to disk, from first boot forward
- Requires no user intervention for providing passphrases
- Uses AES-256-XTS encryption

1.4.1.1. Configuring an encryption threshold

In OpenShift Container Platform, you can specify a requirement for more than one Tang server. You can also configure the TPM v2 and Tang encryption modes simultaneously. This enables boot disk data

decryption only if the TPM secure cryptoprocessor is present and the Tang servers are accessible over a secure network.

You can use the **threshold** attribute in your Butane configuration to define the minimum number of TPM v2 and Tang encryption conditions required for decryption to occur.

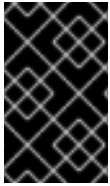
The threshold is met when the stated value is reached through any combination of the declared conditions. In the case of offline provisioning, the offline server is accessed using an included advertisement, and only uses that supplied advertisement if the number of online servers do not meet the set threshold.

For example, the **threshold** value of **2** in the following configuration can be reached by accessing two Tang servers, with the offline server available as a backup, or by accessing the TPM secure cryptoprocessor and one of the Tang servers:

Example Butane configuration for disk encryption

```
variant: openshift
version: 4.20.0
metadata:
  name: worker-storage
  labels:
    machineconfiguration.openshift.io/role: worker
boot_device:
  layout: x86_64 ❶
  luks:
    tpm2: true ❷
    tang: ❸
      - url: http://tang1.example.com:7500
        thumbprint: jwGN5tRFK-kF6pIX89ssF3khxxX
      - url: http://tang2.example.com:7500
        thumbprint: VCJsvZFjBSIHSlw78rOrq7h2ZF
      - url: http://tang3.example.com:7500
        thumbprint: PLjNyRdGw03zIRoGjQYMahSZGu9
      advertisement: "{\"payload\": \"...\", \"protected\": \"...\", \"signature\": \"...\"}" ❹
    threshold: 2 ❺
openshift:
  fips: true
```

- ❶ Set this field to the instruction set architecture of the cluster nodes. Some examples include, **x86_64**, **aarch64**, or **ppc64le**.
- ❷ Include this field if you want to use a Trusted Platform Module (TPM) to encrypt the root file system.
- ❸ Include this section if you want to use one or more Tang servers.
- ❹ Optional: Include this field for offline provisioning. Ignition will provision the Tang server binding rather than fetching the advertisement from the server at runtime. This lets the server be unavailable at provisioning time.
- ❺ Specify the minimum number of TPM v2 and Tang encryption conditions required for decryption to occur.



IMPORTANT

The default **threshold** value is **1**. If you include multiple encryption conditions in your configuration but do not specify a threshold, decryption can occur if any of the conditions are met.



NOTE

If you require TPM v2 *and* Tang for decryption, the value of the **threshold** attribute must equal the total number of stated Tang servers plus one. If the **threshold** value is lower, it is possible to reach the threshold value by using a single encryption mode. For example, if you set **tpm2** to **true** and specify two Tang servers, a threshold of **2** can be met by accessing the two Tang servers, even if the TPM secure cryptoprocessor is not available.

1.4.2. About disk mirroring

During OpenShift Container Platform installation on control plane and worker nodes, you can enable mirroring of the boot and other disks to two or more redundant storage devices. A node continues to function after storage device failure provided one device remains available.

Mirroring does not support replacement of a failed disk. Reprovision the node to restore the mirror to a pristine, non-degraded state.



NOTE

For user-provisioned infrastructure deployments, mirroring is available only on RHCOS systems. Support for mirroring is available on **x86_64** nodes booted with BIOS or UEFI and on **ppc64le** nodes.

1.4.3. Configuring disk encryption and mirroring

You can enable and configure encryption and mirroring during an OpenShift Container Platform installation.

Prerequisites

- You have downloaded the OpenShift Container Platform installation program on your installation node.
- You installed Butane on your installation node.



NOTE

Butane is a command-line utility that OpenShift Container Platform uses to offer convenient, short-hand syntax for writing and validating machine configs. For more information, see "Creating machine configs with Butane".

- You have access to a Red Hat Enterprise Linux (RHEL) 8 machine that can be used to generate a thumbprint of the Tang exchange key.

Procedure

1. If you want to use TPM v2 to encrypt your cluster, check to see if TPM v2 encryption needs to be enabled in the host firmware for each node. This is required on most Dell systems. Check the manual for your specific system.
2. If you want to use Tang to encrypt your cluster, follow these preparatory steps:
 - a. Set up a Tang server or access an existing one. See [Network-bound disk encryption](#) for instructions.
 - b. Install the **clevis** package on a RHEL 8 machine, if it is not already installed:

```
$ sudo yum install clevis
```

- c. On the RHEL 8 machine, run the following command to generate a thumbprint of the exchange key. Replace **http://tang1.example.com:7500** with the URL of your Tang server:

```
$ clevis-encrypt-tang '{"url":"http://tang1.example.com:7500"}' < /dev/null > /dev/null 1
```

¹ In this example, **tangd.socket** is listening on port **7500** on the Tang server.



NOTE

The **clevis-encrypt-tang** command generates a thumbprint of the exchange key. No data passes to the encryption command during this step; **/dev/null** exists here as an input instead of plain text. The encrypted output is also sent to **/dev/null**, because it is not required for this procedure.

Example output

The advertisement contains the following signing keys:

```
PLjNyRdGw03zIRoGjQYMahSZGu9 1
```

¹ The thumbprint of the exchange key.

When the **Do you wish to trust these keys? [ynYN]** prompt displays, type **Y**.

- d. Optional: For offline Tang provisioning:
 - i. Obtain the advertisement from the server using the **curl** command. Replace **http://tang2.example.com:7500** with the URL of your Tang server:

```
$ curl -f http://tang2.example.com:7500/adv > adv.jws && cat adv.jws
```

Expected output

```
{"payload": "eyJrZXlzljogW3siYWxnljogIkV", "protected":  
"eyJhbGciOiJFUzUxMjI0IiwiaXNjaWkiOiJkaWki", "signature": "ADLgk7fZdE3Yt4FyYsm0pHiau7Q"}
```

- ii. Provide the advertisement file to Clevis for encryption:

```
$ clevis-encrypt-tang '{"url":"http://tang2.example.com:7500","adv":"adv.jws"}' <
/dev/null > /dev/null
```

- e. If the nodes are configured with static IP addressing, run **coreos-installer iso customize --dest-karg-append** or use the **coreos-installer --append-karg** option when installing RHCOS nodes to set the IP address of the installed system. Append the **ip=** and other arguments needed for your network.



IMPORTANT

Some methods for configuring static IPs do not affect the initramfs after the first boot and will not work with Tang encryption. These include the **coreos-installer --copy-network** option, the **coreos-installer iso customize --network-keyfile** option, and the **coreos-installer pxe customize --network-keyfile** option, as well as adding **ip=** arguments to the kernel command line of the live ISO or PXE image during installation. Incorrect static IP configuration causes the second boot of the node to fail.

3. On your installation node, change to the directory that contains the installation program and generate the Kubernetes manifests for the cluster:

```
$ ./openshift-install create manifests --dir <installation_directory> ❶
```

- ❶ Replace **<installation_directory>** with the path to the directory that you want to store the installation files in.

4. Create a Butane config that configures disk encryption, mirroring, or both. For example, to configure storage for compute nodes, create a **\$HOME/clusterconfig/worker-storage.bu** file.

Butane config example for a boot device

```
variant: openshift
version: 4.20.0
metadata:
  name: worker-storage ❶
  labels:
    machineconfiguration.openshift.io/role: worker ❷
boot_device:
  layout: x86_64 ❸
  luks: ❹
  tpm2: true ❺
  tang: ❻
    - url: http://tang1.example.com:7500 ❼
      thumbprint: PLjNyRdGw03zIRoGjQYMahSZGu9 ❽
    - url: http://tang2.example.com:7500
      thumbprint: VCJsvZFjBSIHsldw78rOrq7h2ZF
      advertisement: '{"payload": "eyJrZXlzljogW3siYWxnljogIkV", "protected":
"eyJhbGciOiJIJFZlbnN0eSI", "signature": "ADLgk7fZdE3Yt4FyYsm0pHiau7Q"}' ❾
  threshold: 1 ❿
  mirror: 11
  devices: 12
```

```

- /dev/sda
- /dev/sdb
openshift:
  fips: true 13

```

- 1** **2** For control plane configurations, replace **worker** with **master** in both of these locations.
- 3** Set this field to the instruction set architecture of the cluster nodes. Some examples include, **x86_64**, **aarch64**, or **ppc64le**.
- 4** Include this section if you want to encrypt the root file system. For more details, see "About disk encryption".
- 5** Include this field if you want to use a Trusted Platform Module (TPM) to encrypt the root file system.
- 6** Include this section if you want to use one or more Tang servers.
- 7** Specify the URL of a Tang server. In this example, **tangd.socket** is listening on port **7500** on the Tang server.
- 8** Specify the exchange key thumbprint, which was generated in a preceding step.
- 9** Optional: Specify the advertisement for your offline Tang server in valid JSON format.
- 10** Specify the minimum number of TPM v2 and Tang encryption conditions that must be met for decryption to occur. The default value is **1**. For more information about this topic, see "Configuring an encryption threshold".
- 11** Include this section if you want to mirror the boot disk. For more details, see "About disk mirroring".
- 12** List all disk devices that should be included in the boot disk mirror, including the disk that RHCOS will be installed onto.
- 13** Include this directive to enable FIPS mode on your cluster.



IMPORTANT

To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Installing the system in FIPS mode](#). If you are configuring nodes to use both disk encryption and mirroring, both features must be configured in the same Butane configuration file. If you are configuring disk encryption on a node with FIPS mode enabled, you must include the **fips** directive in the same Butane configuration file, even if FIPS mode is also enabled in a separate manifest.

5. Create a control plane or compute node manifest from the corresponding Butane configuration file and save it to the **<installation_directory>/openshift** directory. For example, to create a manifest for the compute nodes, run the following command:

```

$ butane $HOME/clusterconfig/worker-storage.bu -o <installation_directory>/openshift/99-
worker-storage.yaml

```

Repeat this step for each node type that requires disk encryption or mirroring.

6. If you enable encryption, edit the manifest that was produced by the previous step and replace the cipher **aes-cbc-essiv:sha256** with **aes-xts-plain64**. The following excerpt shows a sample encryption configuration after this change:

```
# ...
luks:
# ...
  options:
    - --cipher
    - aes-xts-plain64
```

7. Save the Butane configuration file in case you need to update the manifests in the future.
8. Continue with the remainder of the OpenShift Container Platform installation.

TIP

You can monitor the console log on the RHCOS nodes during installation for error messages relating to disk encryption or mirroring.



IMPORTANT

If you configure additional data partitions, they will not be encrypted unless encryption is explicitly requested.

Verification

After installing OpenShift Container Platform, you can verify if boot disk encryption or mirroring is enabled on the cluster nodes.

1. From the installation host, access a cluster node by using a debug pod:
 - a. Start a debug pod for the node, for example:
- b. Set **/host** as the root directory within the debug shell. The debug pod mounts the root file system of the node in **/host** within the pod. By changing the root directory to **/host**, you can run binaries contained in the executable paths on the node:

```
$ oc debug node/compute-1
```

```
# chroot /host
```



NOTE

OpenShift Container Platform cluster nodes running Red Hat Enterprise Linux CoreOS (RHCOS) are immutable and rely on Operators to apply cluster changes. Accessing cluster nodes using SSH is not recommended. However, if the OpenShift Container Platform API is not available, or **kubelet** is not properly functioning on the target node, **oc** operations will be impacted. In such situations, it is possible to access nodes using **ssh core@<node>.<cluster_name>.<base_domain>** instead.

2. If you configured boot disk encryption, verify if it is enabled:

- a. From the debug shell, review the status of the root mapping on the node:

```
# cryptsetup status root
```

Example output

```
/dev/mapper/root is active and is in use.
type: LUKS2 ❶
cipher: aes-xts-plain64 ❷
keysize: 512 bits
key location: keyring
device: /dev/sda4 ❸
sector size: 512
offset: 32768 sectors
size: 15683456 sectors
mode: read/write
```

- ❶ The encryption format. When the TPM v2 or Tang encryption modes are enabled, the RHCOS boot disks are encrypted using the LUKS2 format.
- ❷ The encryption algorithm used to encrypt the LUKS2 volume.
- ❸ The device that contains the encrypted LUKS2 volume. If mirroring is enabled, the value will represent a software mirror device, for example **/dev/md126**.

- b. List the Clevis plugins that are bound to the encrypted device:

```
# clevis luks list -d /dev/sda4 ❶
```

- ❶ Specify the device that is listed in the **device** field in the output of the preceding step.

Example output

```
1: sss '{"t":1,"pins":{"tang":{"url":"http://tang.example.com:7500"}}}' ❶
```

- ❶ In the example output, the Tang plugin is used by the Shamir's Secret Sharing (SSS) Clevis plugin for the **/dev/sda4** device.

3. If you configured mirroring, verify if it is enabled:

- a. From the debug shell, list the software RAID devices on the node:

```
# cat /proc/mdstat
```

Example output

```
Personalities : [raid1]
md126 : active raid1 sdb3[1] sda3[0] ❶
```



```

393152 blocks super 1.0 [2/2] [UU]

md127 : active raid1 sda4[0] sdb4[1] 2
51869632 blocks super 1.2 [2/2] [UU]

unused devices: <none>

```

- 1 The **/dev/md126** software RAID mirror device uses the **/dev/sda3** and **/dev/sdb3** disk devices on the cluster node.
- 2 The **/dev/md127** software RAID mirror device uses the **/dev/sda4** and **/dev/sdb4** disk devices on the cluster node.

- b. Review the details of each of the software RAID devices listed in the output of the preceding command. The following example lists the details of the **/dev/md126** device:

```
# mdadm --detail /dev/md126
```

Example output

```

/dev/md126:
  Version : 1.0
  Creation Time : Wed Jul 7 11:07:36 2021
  Raid Level : raid1 1
  Array Size : 393152 (383.94 MiB 402.59 MB)
  Used Dev Size : 393152 (383.94 MiB 402.59 MB)
  Raid Devices : 2
  Total Devices : 2
  Persistence : Superblock is persistent

  Update Time : Wed Jul 7 11:18:24 2021
  State : clean 2
  Active Devices : 2 3
  Working Devices : 2 4
  Failed Devices : 0 5
  Spare Devices : 0

Consistency Policy : resync

  Name : any:md-boot 6
  UUID : ccfa3801:c520e0b5:2bee2755:69043055
  Events : 19

  Number Major Minor RaidDevice State
    0   252    3    0   active sync  /dev/sda3 7
    1   252   19    1   active sync  /dev/sdb3 8

```

- 1 Specifies the RAID level of the device. **raid1** indicates RAID 1 disk mirroring.
- 2 Specifies the state of the RAID device.
- 3 4 States the number of underlying disk devices that are active and working.

- 5 States the number of underlying disk devices that are in a failed state.
- 6 The name of the software RAID device.
- 7 8 Provides information about the underlying disk devices used by the software RAID device.

c. List the file systems mounted on the software RAID devices:

```
# mount | grep /dev/md
```

Example output

```
/dev/md127 on / type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /etc type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /usr type xfs
(ro,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /sysroot type xfs
(ro,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/containers/storage/overlay type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/kubelet/pods/e5054ed5-f882-4d14-b599-99c050d4e0c0/volume-
subpaths/etc/tuned/1 type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/kubelet/pods/e5054ed5-f882-4d14-b599-99c050d4e0c0/volume-
subpaths/etc/tuned/2 type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/kubelet/pods/e5054ed5-f882-4d14-b599-99c050d4e0c0/volume-
subpaths/etc/tuned/3 type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/kubelet/pods/e5054ed5-f882-4d14-b599-99c050d4e0c0/volume-
subpaths/etc/tuned/4 type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md127 on /var/lib/kubelet/pods/e5054ed5-f882-4d14-b599-99c050d4e0c0/volume-
subpaths/etc/tuned/5 type xfs
(rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,prjquota)
/dev/md126 on /boot type ext4 (rw,relatime,seclabel)
```

In the example output, the **/boot** file system is mounted on the **/dev/md126** software RAID device and the root file system is mounted on **/dev/md127**.

4. Repeat the verification steps for each OpenShift Container Platform node type.

Additional resources

- For more information about the TPM v2 and Tang encryption modes, see [Configuring automated unlocking of encrypted volumes using policy-based decryption](#).

1.5. CONFIGURING A RAID-ENABLED DATA VOLUME

You can enable software Redundant Array of Independent Disks (RAID) partitioning to provide an external data volume.

OpenShift Container Platform supports RAID 0, RAID 1, RAID 4, RAID 5, RAID 6, and RAID 10 for data protection and fault tolerance. See "About disk mirroring" for more details.



NOTE

OpenShift Container Platform 4.20 supports manually configuring a hybrid RAID on an installation drive. For a manually configured example, see "Configuring an Intel® Virtual RAID on CPU (VROC) data volume".

Prerequisites

- You have downloaded the OpenShift Container Platform installation program on your installation node.
- You have installed Butane on your installation node.



NOTE

Butane is a command-line utility that OpenShift Container Platform uses to write machine configs. The utility provides convenient, short-hand syntax for writing machine configs and for performing additional validation of machine configs. For more information, see the *Creating machine configs with Butane* section.

Procedure

1. Create a Butane config that configures a data volume by using software RAID.
 - To configure a data volume with RAID 1 on the same disks that are used for a mirrored boot disk, create a **\$HOME/clusterconfig/raid1-storage.bu** file:

Example configuration for RAID 1 on a mirrored boot disk

```
variant: openshift
version: 4.20.0
metadata:
  name: raid1-storage
  labels:
    machineconfiguration.openshift.io/role: worker
boot_device:
  mirror:
    devices:
      - /dev/disk/by-id/scsi-3600508b400105e210000900000490000
      - /dev/disk/by-id/scsi-SSEAGATE_ST373453LW_3HW1RHM6
storage:
  disks:
    - device: /dev/disk/by-id/scsi-3600508b400105e210000900000490000
      partitions:
        - label: root-1
          size_mib: 25000
        - label: var-1
    - device: /dev/disk/by-id/scsi-SSEAGATE_ST373453LW_3HW1RHM6
      partitions:
```

```

    - label: root-2
      size_mib: 25000
    - label: var-2
  raid:
    - name: md-var
      level: raid1
      devices:
        - /dev/disk/by-partlabel/var-1
        - /dev/disk/by-partlabel/var-2
  filesystems:
    - device: /dev/md/md-var
      path: /var
      format: xfs
      wipe_filesystem: true
      with_mount_unit: true
# ...

```

The **size_mib** field adds a data partition to the boot disk. A minimum value of 25000 mebibytes is recommended. If no value is specified or the specified value is smaller than the recommended minimum, the resulting root file system will be too small. Future reinstalls of RHCOS might overwrite the beginning of the data partition.

- To configure a data volume with RAID 1 on secondary disks, create a **\$HOME/clusterconfig/raid1-alt-storage.bu** file:

Example configuration for RAID 1 on secondary disks

```

variant: openshift
version: 4.20.0
metadata:
  name: raid1-alt-storage
  labels:
    machineconfiguration.openshift.io/role: worker
storage:
  disks:
    - device: /dev/sdc
      wipe_table: true
      partitions:
        - label: data-1
    - device: /dev/sdd
      wipe_table: true
      partitions:
        - label: data-2
  raid:
    - name: md-var-lib-containers
      level: raid1
      devices:
        - /dev/disk/by-partlabel/data-1
        - /dev/disk/by-partlabel/data-2
  filesystems:
    - device: /dev/md/md-var-lib-containers
      path: /var/lib/containers
      format: xfs
      wipe_filesystem: true
      with_mount_unit: true
# ...

```

2. Create a RAID manifest from the Butane config. Save the config to the **<installation_directory>/openshift** directory. For example, to create a manifest for the compute nodes, run the following command:

```
$ butane $HOME/clusterconfig/<butane_config>.bu -o
<installation_directory>/openshift/<manifest_name>.yaml
```

Replace the **<manifest_name>** and **<butane_config>** values with the file names from a previous step. For example, **raid1-alt-storage.bu** and **raid1-alt-storage.yaml** for secondary disks.

3. Save the Butane config in case you need to update the manifest in the future.
4. Continue with the remainder of the OpenShift Container Platform installation.

1.5.1. Configuring an Intel® Virtual RAID on CPU (VROC) data volume

Intel® VROC is a type of hybrid RAID, where some of the maintenance is offloaded to the hardware, but appears as software RAID to the operating system.

The following procedure configures an Intel® VROC-enabled RAID1.

Prerequisites

- You have a system with Intel® Volume Management Device (VMD) enabled.

Procedure

1. Create the Intel® Matrix Storage Manager (IMSM) RAID container by running the following command:

```
$ mdadm -CR /dev/md/ims0 -e \
  imsm -n2 /dev/nvme0n1 /dev/nvme1n1 1
```

- 1** The RAID device names. In this example, there are two devices listed. If you provide more than two device names, you must adjust the **-n** flag. For example, listing three devices would use the flag **-n3**.

2. Create the RAID1 storage inside the container:
 - a. Create a dummy RAID0 volume in front of the real RAID1 volume by running the following command:

```
$ mdadm -CR /dev/md/dummy -l0 -n2 /dev/md/ims0 -z10M --assume-clean
```

- b. Create the real RAID1 array by running the following command:

```
$ mdadm -CR /dev/md/coreos -l1 -n2 /dev/md/ims0
```

- c. Stop both RAID0 and RAID1 member arrays and delete the dummy RAID0 array with the following commands:

```
$ mdadm -S /dev/md/dummy \
mdadm -S /dev/md/coreos \
mdadm --kill-subarray=0 /dev/md/imsm0
```

- d. Restart the RAID1 arrays by running the following command:

```
$ mdadm -A /dev/md/coreos /dev/md/imsm0
```

3. Install RHCOS on the RAID1 device:

- a. Get the UUID of the IMSM container by running the following command:

```
$ mdadm --detail --export /dev/md/imsm0
```

- b. Install RHCOS and include the **rd.md.uuid** kernel argument by running the following command:

```
$ coreos-installer install /dev/md/coreos \
--append-karg rd.md.uuid=<md_UUID> 1
...
```

1 The UUID of the IMSM container.

Include any additional **coreos-installer** arguments you need to install RHCOS.

1.6. CONFIGURING CHRONY TIME SERVICE

You can set the time server and related settings used by the chrony time service (**chronyd**) by modifying the contents of the **chrony.conf** file and passing those contents to your nodes as a machine config.

Procedure

1. Create a Butane config including the contents of the **chrony.conf** file. For example, to configure chrony on worker nodes, create a **99-worker-chrony.bu** file.



NOTE

The [Butane version](#) you specify in the config file should match the OpenShift Container Platform version and always ends in **0**. For example, **4.20.0**. See "Creating machine configs with Butane" for information about Butane.

```
variant: openshift
version: 4.20.0
metadata:
  name: 99-worker-chrony
  labels:
    machineconfiguration.openshift.io/role: worker
storage:
  files:
    - path: /etc/chrony.conf
```

```

mode: 0644
overwrite: true
contents:
  inline: |
    pool 0.rhel.pool.ntp.org iburst
    driftfile /var/lib/chrony/drift
    makestep 1.0 3
    rtcsync
    logdir /var/log/chrony

```

- **name: 99-worker-chrony** - Specify a name for the machine config file. On control plane nodes, substitute **master** for **worker**.
- **machineconfiguration.openshift.io/role: worker** - On control plane nodes, substitute **master** for **worker**.
- **mode: 0644** - Specify an octal value mode for the **mode** field in the machine config file. After creating the file and applying the changes, the **mode** is converted to a decimal value. You can check the YAML file with the command **oc get mc <mc-name> -o yaml**.
- **pool 0.rhel.pool.ntp.org iburst** - Specify any valid, reachable time source, such as the one provided by your DHCP server.



NOTE

For all-machine to all-machine communication, the Network Time Protocol (NTP) on UDP is port **123**. If an external NTP time server is configured, you must open UDP port **123**.

Alternatively, you can specify any of the following NTP servers: **1.rhel.pool.ntp.org**, **2.rhel.pool.ntp.org**, or **3.rhel.pool.ntp.org**. When you use NTP with your DHCP server, you must set the **sourcedir /run/chrony-dhcp** parameter in the **chrony.conf** file.

2. Use Butane to generate a **MachineConfig** object file, **99-worker-chrony.yaml**, containing the configuration to be delivered to the nodes:

```
$ butane 99-worker-chrony.bu -o 99-worker-chrony.yaml
```

3. Apply the configurations in one of two ways:
 - If the cluster is not running yet, after you generate manifest files, add the **MachineConfig** object file to the **<installation_directory>/openshift** directory, and then continue to create the cluster.
 - If the cluster is already running, apply the file:

```
$ oc apply -f ./99-worker-chrony.yaml
```

For more information on chrony best practices, see the following resources:

- [Configuring chrony](#)
- [Best practices for NTP](#)

- [Basic chrony NTP troubleshooting](#)

1.7. ADDITIONAL RESOURCES

- For information on Butane, see [Creating machine configs with Butane](#).
- For information on FIPS support, see [Support for FIPS cryptography](#).

CHAPTER 2. CONFIGURING YOUR FIREWALL

If you use a firewall, you must configure it so that OpenShift Container Platform can access the sites that it requires to function. You must always grant access to some sites, and you grant access to more if you use Red Hat Lightspeed, the Telemetry service, a cloud to host your cluster, and certain build strategies.

2.1. CONFIGURING YOUR FIREWALL FOR OPENSIFT CONTAINER PLATFORM

Before you install OpenShift Container Platform, you must configure your firewall to grant access to the sites that OpenShift Container Platform requires. When using a firewall, make additional configurations to the firewall so that OpenShift Container Platform can access the sites that it requires to function.

There are no special configuration considerations for services running on only controller nodes compared to compute nodes.



NOTE

If your environment has a dedicated load balancer in front of your OpenShift Container Platform cluster, review the allowlists between your firewall and load balancer to prevent unwanted network restrictions to your cluster.

Procedure

1. Allowlist the following container registry URLs for cluster installation and upgrades:

URL	Port	Function
registry.redhat.io	443	Provides core container images
access.redhat.com	443	Hosts a signature store that a container client requires for verifying images pulled from registry.access.redhat.com . In a firewall environment, ensure that this resource is on the allowlist.
registry.access.redhat.com	443	Hosts all the container images that are stored on the Red Hat Ecosystem Catalog, including core container images.
quay.io	443	Provides core container images
cdn.quay.io	443	Provides core container images
cdn01.quay.io	443	Provides core container images
cdn02.quay.io	443	Provides core container images
cdn03.quay.io	443	Provides core container images

URL	Port	Function
cdn04.quay.io	443	Provides core container images
cdn05.quay.io	443	Provides core container images
cdn06.quay.io	443	Provides core container images
icr.io	443	Provides IBM Cloud Pak container images. This domain is only required if you use IBM Cloud Paks.
cp.icr.io	443	Provides IBM Cloud Pak container images. This domain is only required if you use IBM Cloud Paks.

- You can use the wildcard ***.quay.io** instead of **cdn.quay.io** and **cdn0[1-6].quay.io** in your allowlist.
- You can use the wildcard ***.access.redhat.com** to simplify the configuration and ensure that all subdomains, including **registry.access.redhat.com**, are allowed.
- When adding a site such as **quay.io** to your allowlist, do not add a wildcard entry such as ***.quay.io** to your denylist. In most cases, image registries use a content delivery network (CDN) to serve images. If a firewall blocks access, image downloads are denied when the initial download request redirects to a hostname such as **cdn01.quay.io**.

2. Allowlist the following URLs to enable cluster access, authentication, and updates:

URL	Port	Function
*.apps.<cluster_name>.<base_domain>	443	Allowlist these URLs to enable cluster access, authentication, and updates.
api.openshift.com	443	API endpoint for cluster tokens and update checks.
console.redhat.com	443	Authentication service for cluster tokens.
sso.redhat.com	443	The https://console.redhat.com site uses authentication from sso.redhat.com

For egress traffic, Operators require route access to perform health checks to establish a connection for reaching endpoints. The authentication and web console Operators connect to two routes to verify functionality. Cluster administrators who do not want to allow ***.apps.<cluster_name>.<base_domain>**, must allow the following routes:

- **oauth-openshift.apps.<cluster_name>.<base_domain>**

- **canary-openshift-ingress-canary.apps.<cluster_name>.<base_domain>**
 - **console-openshift-console.apps.<cluster_name>.<base_domain>**, or the hostname that is specified in the **spec.route.hostname** field of the **consoles.operator/cluster** object if the field is not empty.
3. Allowlist the following registry URLs that host related artifacts for cluster installation and upgrades, such as installation content, release images, and client tools:

URL	Port	Function
mirror.openshift.com	443	Required to access mirrored installation content and images. This site is also a source of release image signatures, although the Cluster Version Operator needs only a single functioning source.
quayio-production-s3.s3.amazonaws.com	443	Required to access Quay image content in AWS.
rhcos.mirror.openshift.com	443	Required to download Red Hat Enterprise Linux CoreOS (RHCOS) images.
storage.googleapis.com/openshift-release	443	A source of release image signatures, although the Cluster Version Operator needs only a single functioning source.

4. Set your firewall's allowlist to include any site that provides resources for a language or framework that your builds require.
5. If you do not disable Telemetry, you must grant access to the following URLs to access Telemetry and Red Hat Lightspeed:

URL	Port	Function
cert-api.access.redhat.com	443	Required for Telemetry
api.access.redhat.com	443	Required for Telemetry
infogw.api.openshift.com	443	Required for Telemetry
console.redhat.com	443	Required for Telemetry and for insights-operator

6. If you use Alibaba Cloud, Amazon Web Services (AWS), Microsoft Azure, or Google Cloud to host your cluster, you must grant access to the URLs that offer the cloud provider API and DNS for that cloud:

Cloud	URL	Port	Function
Alibaba	*.aliyuncs.com	443	Required to access Alibaba Cloud services and resources. Review the Alibaba endpoints_config.go file to find the exact endpoints to allow for the regions that you use.
AWS	aws.amazon.com	443	Used to install and manage clusters in an AWS environment.
	*.amazonaws.com Alternatively, if you choose to not use a wildcard for AWS APIs, you must include the following URLs in your allowlist:	443	Required to access AWS services and resources. Review the AWS Service Endpoints in the AWS documentation to find the exact endpoints to allow for the regions that you use.
	ec2.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	events.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	iam.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	route53.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	*.s3.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	*.s3. <aws_region>.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	*.s3.dualstack. <aws_region>.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	sts.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	sts. <aws_region>.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	tagging.us-east-1.amazonaws.com	443	Used to install and manage clusters in an AWS environment. This endpoint is always us-east-1 , regardless of the region the cluster is deployed in.

Cloud	URL	Port	Function
	ec2. <aws_region>.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	elasticloadbalancing. <aws_region>.amazonaws.com	443	Used to install and manage clusters in an AWS environment.
	servicequotas. <aws_region>.amazonaws.com	443	Required. Used to confirm quotas for deploying the service.
	tagging. <aws_region>.amazonaws.com	443	Allows the assignment of metadata about AWS resources in the form of tags.
	*.cloudfront.net	443	Used to provide access to CloudFront. If you use the AWS Security Token Service (STS) and the private S3 bucket, you must provide access to CloudFront.
GCP	*.googleapis.com	443	Required to access Google Cloud services and resources. Review Cloud Endpoints in the Google Cloud documentation to find the endpoints to allow for your APIs.
	accounts.google.com	443	Required to access your Google Cloud account.
Microsoft Azure	management.azure.com	443	Required to access Microsoft Azure services and resources. Review the Microsoft Azure REST API reference in the Microsoft Azure documentation to find the endpoints to allow for your APIs.
	*.blob.core.windows.net	443	Required to download Ignition files.
	login.microsoftonline.com	443	Required to access Microsoft Azure services and resources. Review the Azure REST API reference in the Microsoft Azure documentation to find the endpoints to allow for your APIs.

7. Allowlist the following URL for optional third-party content:

URL	Port	Function
registry.connect.redhat.com	443	Required for all third-party images and certified operators.

8. If you use a default Red Hat Network Time Protocol (NTP) server, allow the following URLs. NTP operates on User Datagram Protocol (UDP) port 123, so this port must be opened on the firewall.

URL	Port	Function
1.rhel.pool.ntp.org	123	Provides NTP services for time synchronization.
2.rhel.pool.ntp.org	123	Provides NTP services for time synchronization.
3.rhel.pool.ntp.org	123	Provides NTP services for time synchronization.



NOTE

If you do not use a default Red Hat NTP server, verify the NTP server for your platform and allow it in your firewall.

Additional resources

- [OpenID Connect requirements for AWS STS](#)

2.2. OPENSIFT CONTAINER PLATFORM NETWORK FLOW MATRIX

The following network flow matrixes describe the ingress flows to OpenShift Container Platform services for the following environments:

- OpenShift Container Platform on bare metal
- Single-node OpenShift with other platforms
- OpenShift Container Platform on Amazon Web Services (AWS)
- Single-node OpenShift on AWS

Use the information in the appropriate network flow matrix to help you manage ingress traffic for your specific environment. You can restrict ingress traffic to essential flows to improve network security.

Additionally, consider the following dynamic port ranges when managing ingress traffic for both bare metal and cloud environments:

- **9000-9999**: Reserved for internal OpenShift Container Platform components. Do not assign user workloads or services to ports in this range.

- **30000-32767**: Kubernetes **NodePort** service ports. These ports are required only if you expose services by using the **NodePort** service type. If **NodePort** services are not used, you can block this port range.

To view or download the complete raw CSV content for an environment, see the following resources:

- [OpenShift Container Platform on bare metal](#)
- [Single-node OpenShift with other platforms](#)
- [OpenShift Container Platform on AWS](#)
- [Single-node OpenShift on AWS](#)



NOTE

The network flow matrixes describe ingress traffic flows for a base OpenShift Container Platform or single-node OpenShift installation. The matrixes do not apply for hosted control planes, Red Hat build of MicroShift, or standalone clusters.

2.2.1. Base network flows

The following matrixes describe the base ingress flows to OpenShift Container Platform services.



NOTE

For base ingress flows to single-node OpenShift clusters, see the *Control plane node base flows* matrix only.

Table 2.1. Control plane node base flows

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	22	Host system service	sshd			master	TRUE
Ingress	TCP	111	Host system service	rpcbind			master	TRUE
Ingress	TCP	2379	openshift-etcd	etcd	etcd	etcdctl	master	FALSE
Ingress	TCP	2380	openshift-etcd	healthz	etcd	etcd	master	FALSE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	6080	openshift-kube-apiserver		kube-apiserver	kube-apiserver-insecure-readyz	master	FALSE
Ingress	TCP	6443	openshift-kube-apiserver	apiserver	kube-apiserver	kube-apiserver	master	FALSE
Ingress	TCP	8798	openshift-machine-config-operator	machine-config-daemon	machine-config-daemon	machine-config-daemon	master	FALSE
Ingress	TCP	9001	openshift-machine-config-operator	machine-config-daemon	machine-config-daemon	kube-rbac-proxy	master	FALSE
Ingress	TCP	9099	openshift-cluster-version	cluster-version-operator	cluster-version-operator	cluster-version-operator	master	FALSE
Ingress	TCP	9100	openshift-monitoring	node-exporter	node-exporter	kube-rbac-proxy	master	FALSE
Ingress	TCP	9103	openshift-ovn-kubernetes	ovn-kubernetes-node	ovnkube-node	kube-rbac-proxy-node	master	FALSE
Ingress	TCP	9104	openshift-network-operator	metrics	network-operator	network-operator	master	FALSE
Ingress	TCP	9105	openshift-ovn-kubernetes	ovn-kubernetes-node	ovnkube-node	kube-rbac-proxy-ovn-metrics	master	FALSE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	9107	openshift-ovn-kubernetes	egressip-node-healthcheck	ovnkube-node	ovnkube-controller	master	FALSE
Ingress	TCP	9108	openshift-ovn-kubernetes	ovn-kubernetes-control-plane	ovnkube-control-plane	kube-rbac-proxy	master	FALSE
Ingress	TCP	9192	openshift-cluster-machine-approver	machine-approver	machine-approver	kube-rbac-proxy	master	FALSE
Ingress	TCP	9258	openshift-cloud-controller-manager-operator	machine-approver	cluster-cloud-controller-manager	cluster-cloud-controller-manager	master	FALSE
Ingress	TCP	9637	openshift-machine-config-operator	kube-rbac-proxy-crio	kube-rbac-proxy-crio	kube-rbac-proxy-crio	master	FALSE
Ingress	TCP	9978	openshift-etcd	etcd	etcd	etcd-metrics	master	FALSE
Ingress	TCP	9979	openshift-etcd	etcd	etcd	etcd-metrics	master	FALSE
Ingress	TCP	9980	openshift-etcd	etcd	etcd	etcd	master	FALSE
Ingress	TCP	10250	Host system service	kubelet			master	FALSE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	10256	openshift-ovn-kubernetes	ovnkube	ovnkube	ovnkube-controller	master	FALSE
Ingress	TCP	10257	openshift-kube-controller-manager	kube-controller-manager	kube-controller-manager	kube-controller-manager	master	FALSE
Ingress	TCP	10259	openshift-kube-scheduler	scheduler	openshift-kube-scheduler	kube-scheduler	master	FALSE
Ingress	TCP	17697	openshift-kube-apiserver	openshift-kube-apiserver-healthz	kube-apiserver	kube-apiserver-check-endpoints	master	FALSE
Ingress	TCP	22623	openshift-machine-config-operator	machine-config-server	machine-config-server	machine-config-server	master	FALSE
Ingress	TCP	22624	openshift-machine-config-operator	machine-config-server	machine-config-server	machine-config-server	master	FALSE
Ingress	UDP	111	Host system service	rpcbind			master	TRUE

Table 2.2. Worker node base flows

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	22	Host system service	sshd			worker	TRUE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	111	Host system service	rpcbind			worker	TRUE
Ingress	TCP	8798	openshift-machine-config-operator	machine-config-daemon	machine-config-daemon	machine-config-daemon	worker	FALSE
Ingress	TCP	9001	openshift-machine-config-operator	machine-config-daemon	machine-config-daemon	kube-rbac-proxy	worker	FALSE
Ingress	TCP	9100	openshift-monitoring	node-exporter	node-exporter	kube-rbac-proxy	worker	FALSE
Ingress	TCP	9103	openshift-ovn-kubernetes	ovn-kubernetes-node	ovnkube-node	kube-rbac-proxy-node	worker	FALSE
Ingress	TCP	9105	openshift-ovn-kubernetes	ovn-kubernetes-node	ovnkube-node	kube-rbac-proxy-ovn-metrics	worker	FALSE
Ingress	TCP	9107	openshift-ovn-kubernetes	egressip-node-healthcheck	ovnkube-node	ovnkube-controller	worker	FALSE
Ingress	TCP	9637	openshift-machine-config-operator	kube-rbac-proxy-crio	kube-rbac-proxy-crio	kube-rbac-proxy-crio	worker	FALSE
Ingress	TCP	10250	Host system service	kubelet			worker	FALSE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	10256	openshift-ovn-kubernetes	ovnkube	ovnkube	ovnkube-controller	worker	FALSE
Ingress	UDP	111	Host system service	rpcbind			worker	TRUE

2.2.2. Additional network flows for OpenShift Container Platform on bare metal

In addition to the base network flows, the following matrix describes the ingress flows to OpenShift Container Platform services that are specific to OpenShift Container Platform on bare metal.

Table 2.3. OpenShift Container Platform on bare metal

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	53	openshift-dns	dns-default	dnf-default	dns	master	FALSE
Ingress	TCP	6180	openshift-machine-api	metal3-state	metal3	metal3-httpd	master	FALSE
Ingress	TCP	6183	openshift-machine-api	metal3-state	metal3	metal3-httpd	master	FALSE
Ingress	TCP	6385	openshift-machine-api	metal3-state	metal3	metal3-httpd	master	FALSE
Ingress	TCP	6388	openshift-machine-api	metal3-state	metal3	metal3-httpd	master	FALSE
Ingress	TCP	9444	openshift-kni-infra		haproxy	haproxy	master	FALSE

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	9445	openshift-kni-infra		haproxy	haproxy	master	FALSE
Ingress	TCP	9454	openshift-kni-infra		haproxy	haproxy	master	FALSE
Ingress	TCP	18080	openshift-kni-infra		coredns	coredns	master	FALSE
Ingress	UDP	53	openshift-dns	dns-default	dnf-default	dns	master	FALSE
Ingress	UDP	6081	openshift-ovn-kubernetes	ovn-kubernetes-geneve			master	FALSE
Ingress	TCP	53	openshift-dns	dns-default	dnf-default	dns	worker	FALSE
Ingress	TCP	80	openshift-ingress	router-internal-default	router-default	router	worker	FALSE
Ingress	TCP	443	openshift-ingress	router-internal-default	router-default	router	worker	FALSE
Ingress	TCP	1936	openshift-ingress	router-internal-default	router-default	router	worker	FALSE
Ingress	TCP	18080	openshift-kni-infra		coredns	coredns	worker	FALSE
Ingress	UDP	53	openshift-dns	dns-default	dnf-default	dns	worker	FALSE
Ingress	UDP	6081	openshift-ovn-kubernetes	ovn-kubernetes-geneve			worker	FALSE

2.2.3. Additional network flows for single-node OpenShift with other platforms

In addition to the base network flows, the following matrix describes the ingress flows to OpenShift Container Platform services that are specific to single-node OpenShift configured with **platform: none** in the installation manifest.

Table 2.4. Single-node OpenShift with other platforms

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	80	openshift-ingress	router-internal-default	router-default	router	master	FALSE
Ingress	TCP	443	openshift-ingress	router-internal-default	router-default	router	master	FALSE
Ingress	TCP	1936	openshift-ingress	router-internal-default	router-default	router	master	FALSE

2.2.4. Additional network flows for OpenShift Container Platform on AWS

In addition to the base network flows, the following matrix describes the ingress flows to OpenShift Container Platform services that are specific to OpenShift Container Platform on AWS.

Table 2.5. OpenShift Container Platform on AWS

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	10258	openshift-cloud-controller-manager-operator	cloud-controller	cloud-controller-manager	cloud-controller-manager	master	FALSE
Ingress	TCP	80	openshift-ingress	router-default	router-default	router	worker	FALSE
Ingress	TCP	443	openshift-ingress	router-default	router-default	router	worker	FALSE
Ingress	UDP	6081	openshift-ovn-kubernetes	ovn-kubernetes-geneve			worker	FALSE

2.2.5. Additional network flows for single-node OpenShift on AWS

In addition to the base network flows, the following matrix describes the ingress flows to OpenShift Container Platform services that are specific to single-node OpenShift on AWS.

Table 2.6. Single-node OpenShift on AWS

Direction	Protocol	Port	Namespace	Service	Pod	Container	Node Role	Optional
Ingress	TCP	80	openshift-ingress	router-default	router-default	router	master	FALSE
Ingress	TCP	443	openshift-ingress	router-default	router-default	router	master	FALSE
Ingress	TCP	10258	openshift-cloud-controller-manager-operator	cloud-controller	cloud-controller-manager	cloud-controller-manager	master	FALSE